



**DEPARTEMEN TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI ELEKTRO DAN
INFORMATIKA CERDAS
INSTITUT TEKNOLOGI SEPULUH NOPEMBER**

Kode MK / Course Code : EF234402
Mata Kuliah / Course Name : Konstruksi Perangkat Lunak / Software Construction
Topik / Topic : Case Study – Event Ticketing & Booking System

Event Ticketing & Booking System Using Clean Architecture and Domain-Driven Design

1. Project Overview

In this project, students will develop an Event Ticketing & Booking System. The system allows event organizers to create and publish events, customers to book and pay for tickets, gate officers to validate tickets during check-in, and administrators to manage refund payout.

This project must be implemented using Clean Architecture and Domain-Driven Design tactical patterns. Students are free to choose the programming language, framework, and platform, but the database must use PostgreSQL.

The project will be developed in pairs using a pair programming approach.

2. Learning Objectives

After completing this project, students are expected to be able to:

1. Design software using Clean Architecture with clear separation between domain, application, infrastructure, and presentation layers.
2. Implement domain logic using DDD tactical patterns, including aggregate, entity, value object, domain service, domain event, factory, and repository.
3. Implement use cases using commands, queries, command handlers, and query handlers.

4. Define application service interfaces for external systems and provide their implementations in the infrastructure layer.
5. Implement REST API controllers in the presentation layer.
6. Persist aggregate data using PostgreSQL.
7. Write unit tests for domain logic.
8. Create a ubiquitous language glossary for the problem domain.

3. System Context

The system interacts with several external actors and external systems.

Human Actors:

Event Organizer

The Event Organizer can:

- create events
- manage ticket categories
- publish or cancel events
- approve or reject refund requests
- view sales reports
- view participant lists

Customer

The Customer can:

- browse available events
- view event details
- create ticket bookings
- pay for bookings
- view purchased tickets
- request refunds

Gate Officer

The Gate Officer can:

- validate ticket codes
- check in participants during an event

System Admin

The System Admin can:

- trigger refund payout
- monitor operational processes

External Systems:

The system may interact with:

1. Payment Gateway. Used to process booking payments.
2. Refund Payment Service / Bank Service. Used to process refund payout to customers.
3. Notification Service. Used to send email or WhatsApp notifications.

These external systems must be accessed through application service interfaces defined in the application layer. Their actual implementations must be placed in the infrastructure layer.

4. Required User Stories

Students must implement the following minimum user stories.

Event Management

User Story 1: Create Event

As an Event Organizer, I can create a new event so that tickets can be sold to customers.

Acceptance Criteria

- Given I am authenticated as an Event Organizer, when I enter the event name, description, start date, end date, location, and maximum capacity, then the event is created successfully.
- The event cannot be created if the end date is earlier than the start date.
- The event cannot be created if the maximum capacity is less than or equal to zero.
- A newly created event must have the status Draft.
- After the event is created, the system raises the domain event EventCreated.

User Story 2: Publish Event

As an Event Organizer, I can publish an event so that customers can view the event and purchase tickets.

Acceptance Criteria

- An event can only be published if it has at least one active ticket category.

- An event can only be published if the total ticket quota does not exceed the maximum event capacity.
- An event with the status Draft can be changed to Published.
- An event with the status Cancelled cannot be published.
- After the event is published, the system raises the domain event EventPublished.

User Story 3: Cancel Event

As an Event Organizer, I can cancel an event so that ticket sales are stopped.

Acceptance Criteria

- An event with the status Published can be cancelled.
- An event with the status Completed cannot be cancelled.
- When an event is cancelled, all ticket categories can no longer be purchased.
- Paid bookings must be marked as requiring a refund.
- After the event is cancelled, the system raises the domain event EventCancelled.

Ticket Category Management

User Story 4: Create Ticket Category

As an Event Organizer, I can create ticket categories so that customers can choose from different types of tickets.

Acceptance Criteria

- The Event Organizer can create ticket categories such as Regular, VIP, or Early Bird.
- Each ticket category must have a name, price, quota, sales start date, and sales end date.
- The ticket price cannot be less than zero.
- The ticket quota must be greater than zero.
- The ticket sales period must end before or at the event start date.
- The total quota of all ticket categories must not exceed the maximum event capacity.
- After a ticket category is created, the system raises the domain event TicketCategoryCreated.

User Story 5: Disable Ticket Category

As an Event Organizer, I can disable a ticket category so that customers can no longer purchase tickets from that category.

Acceptance Criteria

- A ticket category can be disabled if the event has not been completed.
- A ticket category that already has bookings must still be stored for historical purposes.
- Customers cannot purchase tickets from an inactive ticket category.
- After a ticket category is disabled, the system raises the domain event `TicketCategoryDisabled`.

Event Browsing and Ticket Booking

User Story 6: View Available Events

As a Customer, I can view available events so that I can choose an event I want to attend.

Acceptance Criteria

- Customers can only view events with the status Published.
- Cancelled events are not displayed in the list of available events.
- Customers can see the event name, date, location, and lowest ticket price.
- Customers can filter events by date or location.

User Story 7: View Event Details

As a Customer, I can view event details so that I can understand the event information and available ticket options.

Acceptance Criteria

- The event detail page displays the event name, description, date, location, organizer, and list of ticket categories.
- Only active ticket categories are displayed for purchase.
- Ticket categories whose sales period has not started are displayed with the status Coming Soon.
- Ticket categories whose sales period has ended are displayed with the status Sales Closed.
- Ticket categories with no remaining quota are displayed with the status Sold Out.

User Story 8: Create Ticket Booking

As a Customer, I can create a ticket booking so that I can reserve tickets before making payment.

Acceptance Criteria

- Customers can select an event, ticket category, and ticket quantity.
 - A booking can only be created for an event with the status Published.
 - A booking can only be created for an active ticket category.
 - A booking can only be created within the ticket sales period.
 - The ticket quantity must be greater than zero.
 - The ticket quantity must not exceed the remaining ticket quota.
 - A customer cannot have more than one active booking for the same event.
 - A newly created booking must have the status PendingPayment.
 - A booking must have a payment deadline, for example 15 minutes after it is created.
 - After a booking is created, the system raises the domain event TicketReserved.
-

User Story 9: Calculate Booking Total Price

As a Customer, I can view the total price of my booking so that I know how much I need to pay.

Acceptance Criteria

- The total price is calculated from the ticket unit price multiplied by the ticket quantity.
- If there is a service fee, the service fee is added to the total price.
- The total price cannot be negative.
- The total price is represented using the value object Money.

Booking Payment

User Story 10: Pay Booking

As a Customer, I can pay for my booking so that my tickets are confirmed.

Acceptance Criteria

- A booking can only be paid if its status is PendingPayment.
- A booking cannot be paid if the payment deadline has passed.

- The payment amount must be equal to the total booking price.
 - After payment is successful, the booking status changes to Paid.
 - After the booking is paid, the system raises the domain event BookingPaid.
 - After the booking is paid, the system issues tickets with unique ticket codes.
-

User Story 11: Expire Booking

As the System, I can mark unpaid bookings as expired after their payment deadline so that reserved ticket quota can be released.

Acceptance Criteria

- A booking with the status PendingPayment changes to Expired after its payment deadline has passed.
- A booking with the status Paid cannot be marked as expired.
- When a booking expires, the previously reserved ticket quota is released.
- After a booking expires, the system raises the domain event BookingExpired.

Ticket and Check-in Management

User Story 12: View Purchased Tickets

As a Customer, I can view my purchased tickets so that I can use them to enter the event.

Acceptance Criteria

- Customers can only view tickets from bookings with the status Paid.
- Each ticket must have a unique ticket code.
- Each ticket must have one of the following statuses: Active, CheckedIn, or Cancelled.
- Tickets from cancelled events must have the status Cancelled or RefundRequired.

User Story 13: Check In Ticket

As a Gate Officer, I can check in a ticket so that the participant can enter the event venue.

Acceptance Criteria

- Check-in can only be performed for the event that matches the ticket.
- The ticket must have the status Active.
- A ticket that has already been checked in cannot be used again.

- Check-in can only be performed on the event day or within the allowed check-in time window.
- After successful check-in, the ticket status changes to CheckedIn.
- After the ticket is checked in, the system raises the domain event TicketCheckedIn.

User Story 14: Reject Invalid Ticket Check-in

As a Gate Officer, I can identify invalid tickets so that fake or duplicate tickets can be rejected.

Acceptance Criteria

- If the ticket code is not found, the system displays a message that the ticket is invalid.
- If the ticket has already been checked in, the system displays a message that the ticket has already been used.
- If the ticket belongs to a different event, the system displays a message that the ticket does not match the event.
- If the event has been cancelled, the system displays a message that the event has been cancelled.
- The ticket status must not change if check-in fails.

Refund Management

User Story 15: Request Refund

As a Customer, I can request a refund so that I can receive my money back according to the refund policy.

Acceptance Criteria

- A refund can only be requested for a booking with the status Paid.
- A refund cannot be requested if any ticket from the booking has already been checked in.
- A refund can only be requested before the refund deadline.
- If the event is cancelled, a refund is automatically allowed.
- A refund must have one of the following statuses: Requested, Approved, Rejected, or PaidOut.
- After a refund is requested, the system raises the domain event RefundRequested.

User Story 16: Approve Refund

As an Event Organizer, I can approve a refund request so that the customer can receive the refund payment.

Acceptance Criteria

- A refund can only be approved if its status is Requested.
- When a refund is approved, its status changes to Approved.
- Related tickets are changed to Cancelled.
- The related booking is changed to Refunded.
- After a refund is approved, the system raises the domain event RefundApproved.

User Story 17: Reject Refund

As an Event Organizer, I can reject a refund request so that invalid refund requests are not processed.

Acceptance Criteria

- A refund can only be rejected if its status is Requested.
- A rejection reason must be provided.
- When a refund is rejected, its status changes to Rejected.
- The related booking remains Paid.
- Related tickets remain Active if they have not been cancelled.
- After a refund is rejected, the system raises the domain event RefundRejected.

User Story 18: Mark Refund as Paid Out

As a System/Admin, I can mark an approved refund as paid out so that the refund process is completed.

Acceptance Criteria

- A refund can only be marked as paid out if its status is Approved.
- A payment reference must be recorded.
- When the refund is paid out, its status changes to PaidOut.
- A paid-out refund cannot be approved, rejected, or cancelled again.
- After the refund is paid out, the system raises the domain event RefundPaidOut.

User Story 19: View Event Sales Report

As an Event Organizer, I can view the event sales report so that I can monitor ticket sales performance.

Acceptance Criteria

- The report displays the number of tickets sold per ticket category.
- The report displays the number of bookings with the statuses PendingPayment, Paid, Expired, and Refunded.
- The report displays the total revenue from paid bookings.

User Story 20: View Event Participants

As an Event Organizer, I can view the participant list so that I can know who is expected to attend the event.

Acceptance Criteria

- The participant list only displays customers from bookings with the status Paid.
- Participants from refunded bookings are not displayed as active participants.
- The participant data includes customer name, ticket category, ticket code, and check-in status.
- The participant list must be retrieved through a query in the application layer.

Ubiquitous Language Glossary

Term	Meaning
Event	An activity organized by an Event Organizer and attended by customers.
Event Organizer	A user who creates and manages events.
Customer	A user who books and purchases tickets.
Gate Officer	A user who validates tickets during event check-in.
Ticket Category	A type of ticket, such as Regular, VIP, or Early Bird.
Quota	The maximum number of tickets available in a ticket category.
Booking	A temporary reservation before payment is completed.
Pending Payment	A booking status indicating that payment has not been completed.
Paid	A booking status indicating that payment has been completed.
Expired	A booking status indicating that the payment deadline has passed.
Ticket	Proof of attendance generated after a booking is paid.
Ticket Code	A unique code used to identify and validate a ticket.

Check-in	The process of validating a ticket when a participant enters the event venue.
Refund	The process of returning money to a customer.
Money	A value object representing an amount and currency.
Sales Period	The period during which a ticket category can be purchased.
Payment Deadline	The deadline for completing payment after a booking is created.

5. Required Tests

Students must write unit tests for the domain layer.

Minimum test cases:

- Event cannot be created with invalid schedule.
- Event cannot be created with zero or negative capacity.
- Event cannot be published without active ticket category.
- Ticket category quota cannot exceed event capacity.
- Booking cannot be created with zero quantity.
- Booking cannot be paid after payment deadline.
- Booking cannot be paid with incorrect payment amount.
- Paid booking cannot expire.
- Checked-in ticket cannot be checked in again.
- Refund cannot be requested if ticket has already been checked in.
- Refund cannot be approved if it is not in Requested status.
- Rejected refund must have a rejection reason.

6. Deliverables

Each team must submit:

- Source code repository
- README file
- Database schema or migration files
- Clean Architecture diagram
- Domain model diagram
- API documentation
- Unit test results
- Short explanation of implemented aggregates and business rules

The README must explain:

- how to run the project
- how to configure PostgreSQL
- how to run database migration
- how to run tests
- list of implemented user stories
- list of implemented domain events
- list of implemented application service interfaces

7. Project Timeline

Week	Activity
Week 7	Case study explanation
Week 8	Progress presentation: project structure
Week 9-10	Progress presentation: domain layer and unit tests
Week 11	Progress presentation: application layer
Week 12	Progress presentation: infrastructure layer
Week 13	Progress presentation: presentation layer

8. Weekly Progress Expectations

Week 8: Project Structure

Students must show:

- clean architecture folder structure
- initial business rules derived from the user stories and acceptance criteria
- initial domain model draft
- initial ubiquitous language glossary

Week 9-10: Domain Layer and Unit Tests

Students must show:

- aggregates
- entities
- value objects
- domain services
- domain events
- repository interfaces
- domain unit tests

Week 11: Application Layer

Students must show:

- commands
- command handlers
- queries
- query handlers
- DTOs
- application service interfaces

Week 12: Infrastructure Layer

Students must show:

- PostgreSQL schema or migrations
- repository implementations
- application service implementations
- database connection configuration

Week 13: Presentation Layer

Students must show:

- REST API controllers
- working endpoints
- request and response examples
- integration between controller, application layer, infrastructure layer, and database

9. Final Note

This project is not a simple CRUD application. Students are expected to demonstrate the ability to design and implement business logic using clean software construction principles. The main focus is not only whether the application works, but also whether the internal structure is maintainable, testable, and aligned with Clean Architecture and Domain-Driven Design.